

Práctica 7: Adquisición de datos de motor y gráfica.

Tyrone Miguel Novillo Bravo
Facultad de Ingeniería
Departamento de Telecomunicaciones
Universidad de Cuenca
tyrone.novillo@ucuenca.edu.ec

Pablo Andrés Bermeo Garcia
Facultad de Ingeniería
Departamento de Telecomunicaciones
Universidad de Cuenca
pablo.bermeo@ucuenca.edu.ec

Resumen

This work presents the implementation and validation of a real-time DC motor speed control system using the Controllino Mega and an HMI Stone interface. The motor driver is actuated via a 5 V PWM signal generated on port X1 (pin D0) of the Controllino, while the encoder pulses (MDC) are counted on digital input IN1 to compute revolutions per minute (RPM). Two hardware interrupts are configured: a timer-based interrupt for RPM calculation and an external interrupt for pulse counting. System identification is performed in MATLAB to obtain a first-order transfer function and PID parameters are derived using the MATLAB PID Tuner. The HMI interface allows dynamic adjustment of the speed reference and PID gains, while displaying real-time plots of reference, control signal, and measured RPM. Experimental results confirm the efficacy of the control strategy and the seamless integration between the Controllino and the HMI.

Index Terms

DC motor control, PWM, encoder feedback, system identification, first-order transfer function, PID tuning, HMI visualization, Controllino Mega

I. INTRODUCCIÓN

El presente trabajo aborda el diseño e implementación de un sistema de control de velocidad para un motor de corriente continua (DC) del Entrenador de Planta de Control (EPC) empleando una señal PWM regulada mediante un slider en una interfaz HMI Stone. Para ello, se utilizó el puerto X1 del Controllino Mega, que entrega señales de 5 V, vinculando el pin D0 al motor DC y el pin IN1 al codificador MDC. El conteo de 36 pulsos por revolución, según especifica el EPC, permite calcular las RPM mediante dos interrupciones en el microcontrolador: una externa para contabilizar los pulsos del encoder y otra por comparación de timer para recalcular periódicamente la velocidad. Adicionalmente, se realizó la identificación del sistema en MATLAB para extraer una función de transferencia de primer orden, y se ajustaron las constantes de un controlador PID con el PID Tuner de MATLAB. Finalmente, la interfaz HMI facilita el ajuste dinámico de la referencia de velocidad y de las ganancias PID, mostrando en tiempo real la señal de control y la velocidad medida, lo que permite validar el desempeño y la robustez del sistema de control embebido.

II. MARCO TEÓRICO

III. MARCO TEÓRICO: CONTROLADOR PID

El controlador PID (Proporcional–Integral–Derivativo) es una de las estrategias de regulación más utilizadas en sistemas de control industrial debido a su sencillez y eficacia. Su objetivo es minimizar el error $e(t)$ entre la referencia $r(t)$ y la salida medida $y(t)$ ajustando la señal de control $u(t)$. La ley de control continua se expresa como:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt},$$

donde:

1. K_p es la ganancia proporcional, responsable de reacciones inmediatas al error presente.
2. K_i es la ganancia integral, que elimina el error estacionario acumulando la suma de errores pasados.
3. K_d es la ganancia derivativa, que predice la tendencia del error y mejora la amortiguación del sistema.

En la práctica embebida, la implementación se realiza de forma discreta e incremental para ejecutarse dentro de interrupciones periódicas. Una forma común de la ley incremental es:

$$\Delta u[k] = K_p (e[k] - e[k-1]) + K_i e[k] + K_d (e[k] - 2e[k-1] + e[k-2]),$$

donde $e[k] = r[k] - y[k]$ es el error en el instante de muestreo k . Este enfoque permite limitar y actualizar la salida en cada ciclo sin necesidad de almacenar integrales o derivadas continuas.

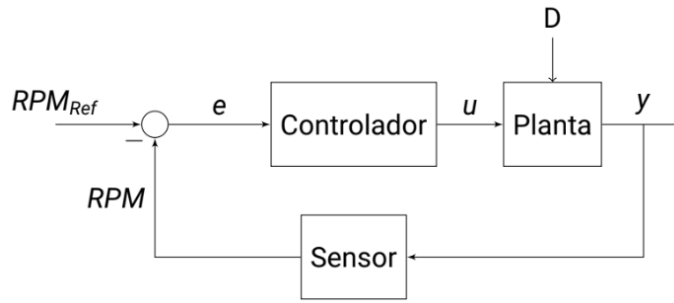


Figura 1. Esquema de control que se empleará en la práctica. Tomado de [?]

En la Figura 1 se ilustra el diagrama de lazo cerrado donde el controlador PID recibe el error e y genera la acción de control u . La correcta selección de K_p , K_i y K_d determina la estabilidad, velocidad de respuesta y amortiguación del sistema controlado.“

IV. DESARROLLO

Esta práctica consiste en controlar la alimentación del motor DC del Entrenador de Planta de Control (EPC) mediante una señal PWM regulada con un slider, al tiempo que se recogen los datos necesarios para calcular su velocidad en RPM y se muestran tanto el duty cycle aplicado como la velocidad del motor en una gráfica del HMI Stone.

El motor del EPC requiere una señal de control de 5 V, pero las salidas digitales del Controllino Mega funcionan a 24 V, por lo que se usa el puerto X1, que opera a 5 V. En concreto, el pin D0 de X1 emite la señal PWM al motor DC, mientras que la salida MDC del encoder del EPC genera pulsos que entran por el pin IN1 de ese mismo puerto.

Según la ficha técnica del EPC, 36 pulsos equivalen a 1 RPM; esta relación se programa en el Controllino Mega para convertir el conteo de pulsos en velocidad.

Para ello se configurarán dos interrupciones en el código de Arduino:

- Una interrupción por timer de comparación que recalcula las RPM del motor.
- Una interrupción externa utilizada para contar los pulsos procedentes de MDC, necesarios para calcular las RPM.

IV-A. Configuración del Hardware

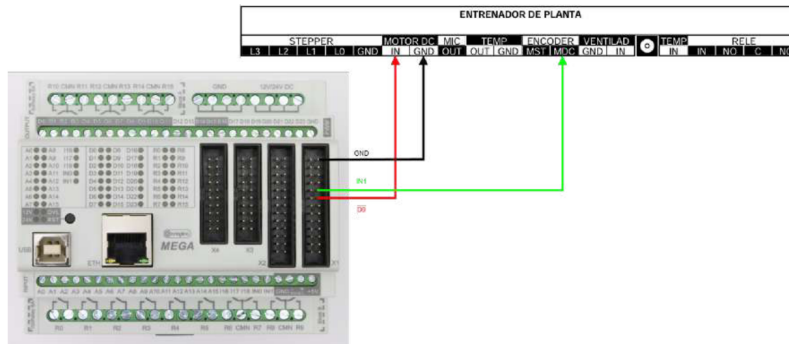


Figura 2. Se presentan las correspondencias entre los botones del tablero y sus entradas con respecto al Controllino.

En la Figura 2 se presentan las configuraciones que se realizaron para comunicar el arduino y el EPC. Allí note que la salida del codificador MDC se enlaza a la entrada digital IN1 del controllino; la salida analógica de voltaje que empleará el controllino para activar el motor parte del D0, por último las referencias GND también se han comunicado. En el apartado (a) de la Figura 2 se visualiza la pantalla HMI Stone que se usará para visualizar las señales de referencia y la velocidad actual del motor.

IV-B. Función de transferencia

Para calcular la función de transferencia, se utilizará un script de Matlab el cual obtiene la función de transferencia de un sistema mediante mediciones de tiempo, referencia y medición real. Para obtener dichas mediciones, se utilizó un programa de Simulink usado en prácticas previas, el cual almacena estos datos en un archivo de texto basado en una simulación. La Figura 3 muestra el esquema del programa que mide la velocidad del motor dc mediante la tarjeta NI myDAQ. De igual forma, se

puede setear un voltaje (referencia) al motor DC. El bucle del programa se repite cada 50ms, esto quiere decir que las muestras guardadas en el archivo están intercaladas cada 50ms.

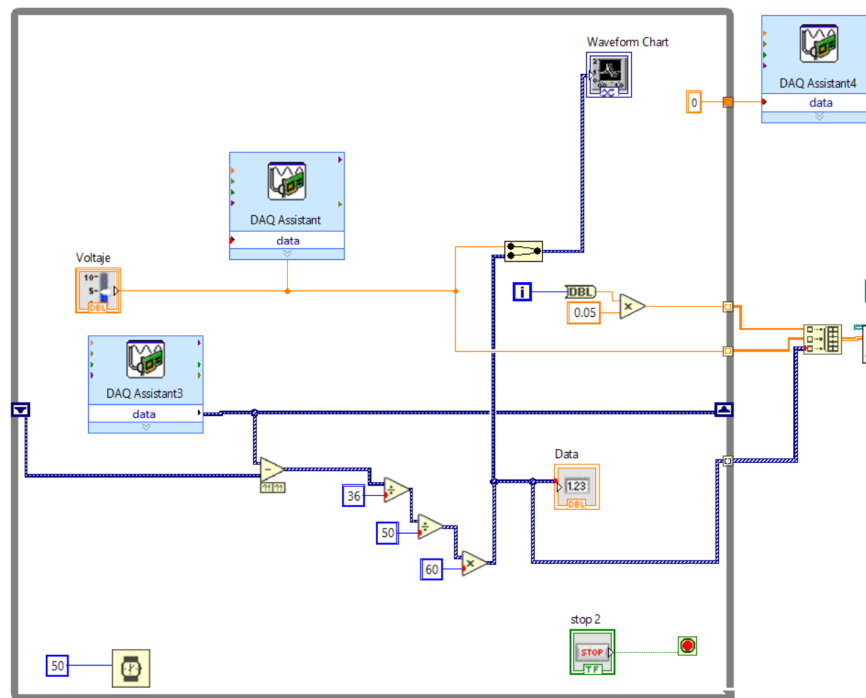


Figura 3. Programa de Labview para guardar datos de tiempo, voltaje de referencia y velocidad en un archivo de texto.

Una vez obtenido el archivo `funciontransf.txt`, se procede a encontrar un modelo de primer orden que represente adecuadamente el comportamiento del sistema a través de matlab.

```
1 datos = readmatrix('funciontransf.txt');
2 tiempo_t = datos(:,1);
3 excitacion_sistema_u = datos(:,2);
4 respuesta_sistema_y = datos(:,3);
```

Inicialmente, se calcula el cambio neto en la entrada (Δu) y en la salida (Δy), lo que permite estimar la ganancia estacionaria del sistema, denotada como k :

```
1 Delta_u = max(excitacion_sistema_u) - min(excitacion_sistema_u);
2 delta_y = max(respuesta_sistema_y) - min(respuesta_sistema_y);
3 k1 = delta_y / Delta_u;
```

El enfoque analítico para obtener la función de transferencia de primer orden con retardo puro consiste en observar la respuesta al escalón y detectar los tiempos en los que la salida alcanza ciertos porcentajes de su valor final. En particular, se consideran los tiempos en los que la señal alcanza el 28.4 % y el 63.2 % del cambio total.

```
1 porcentaje_63 = find(respuesta_sistema_y >= 0.6321*delta_y + Linea_inicio, 1);
2 tiempo_63 = tiempo_t(porcentaje_63);
3 porcentaje_28 = find(respuesta_sistema_y >= 0.284*delta_y + Linea_inicio,1);
```

Se introducen entonces las variables simbólicas `theta3` (retardo) y `tau3` (constante de tiempo), las cuales se resuelven a partir de las siguientes ecuaciones lineales:

- $\theta + \tau = t_{63\%}$
- $\theta + \frac{\tau}{3} = t_{28.4\%}$

Estas ecuaciones se plantean y resuelven usando el toolbox simbólico de MATLAB:

```

1 syms theta3 tau3
2 eq = solve(theta3 + tau3 == tiempo_t(porcentaje_63),...
3           theta3 + tau3/3 == tiempo_t(porcentaje_28));
4 theta3 = eval(eq.theta3);
5 tau3 = eval(eq.tau3);

```

Con los valores de θ_3 y τ_3 ya conocidos, se construye la función de transferencia $G_3(s)$ del sistema utilizando la función `tf` de MATLAB. Esta función de transferencia modela un sistema de primer orden con retardo puro como:

$$G_3(s) = \frac{k_3}{\tau_3 s + 1} e^{-\theta_3 s}$$

```

1 G3 = tf([k3], [tau3 1], 'InputDelay', theta3);
2 [y3, t3] = step(G3);
3 y3 = y3 * Delta_u;

```

El script de matlab indica que la función de transferencia de primer orden para el motor dc es:

$$G(s) = \frac{0,7534}{0,075s + 1}$$

Ahora bien, para obtener los valores de las constantes del controlador PID, se utilizará la herramienta de sintonización PID:

```

1 pidTuner(G3, 'PID');

```

Ajustando los valores de comportamiento trascendental y el tiempo de respuesta, se obtuvieron las siguientes constantes del controlador:

- $K_p = 0$
- $K_i = 0,02103$
- $K_d = 0$

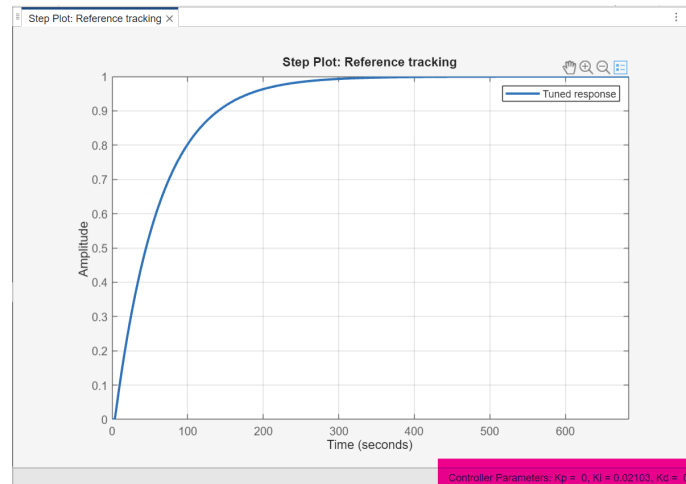


Figura 4. Constantes para el PID Tunner

IV-C. HMI

La Figura 5 muestra la interfaz humano máquina la cual está conformada por los siguientes elementos:

- 3 objetos del tipo spin box para ajustar dinámicamente los valores de las ganancias del controlador PID.
- 1 slider para setear la referencia en revoluciones por minuto del motor
- Un texto informativo que muestra las mediciones del motor en RPM.
- Un objeto del tipo chart view que contiene 3 line series para graficar en tiempo real el valor de la referencia, señal de control y el valor de RPM medido del motor.

Cabe mencionar que los nombres de las variables deben coincidir con las ingresadas posteriormente en el código del motor enviado al Arduino.

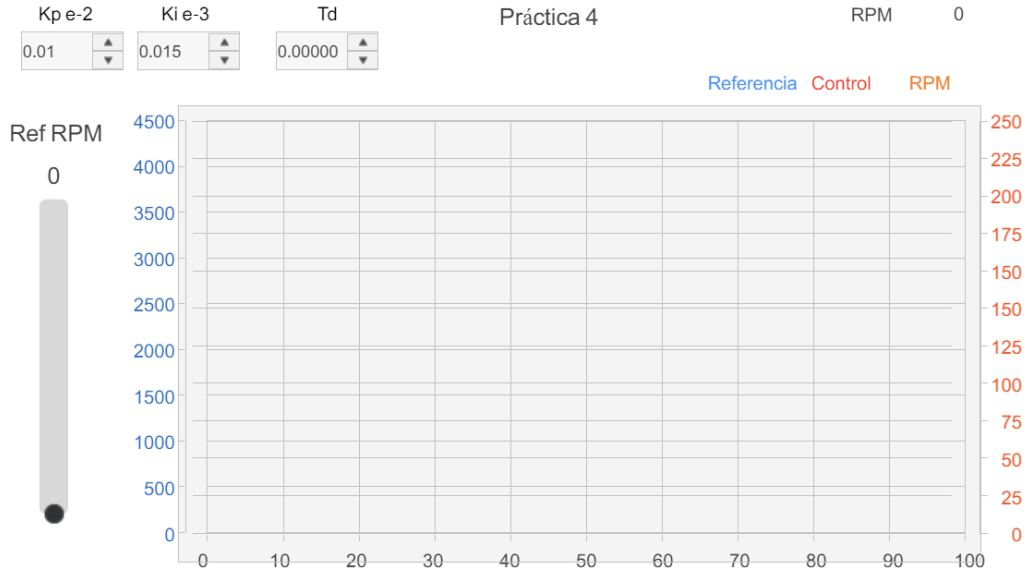


Figura 5. HMI configurada en el software Stone

IV-D. Configuración de Hardware

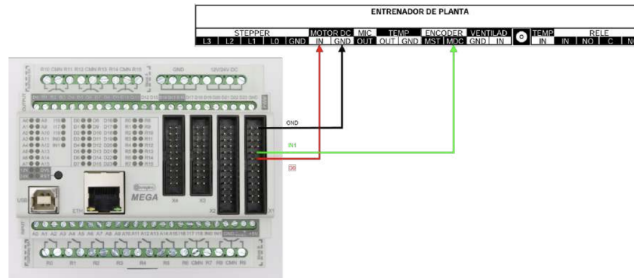


Figura 6. Enter Caption

IV-E. Control del motor DC

La práctica guiada partió de una codificación que permitía, en términos generales, el conteo de pulsos provenientes del motor y su control de velocidad a través de un valor arbitrario PWM. Para conseguir tal fin se emplea una lógica que interrupciones que suceden cada segundo. El código proveído también se encarga de comunicarse con la pantalla a través de las etiquetas correspondientes a objetos generados en su configuración.

La programación del microprocesador no es el tema principal de la práctica, en tanto se realizará una explicación superficial sobre la implementación de la interrupción. No obstante, se hará extensa explicación en la implementación de la ley de control y la comunicación con la pantalla. En cuanto a la interrupción empleada considere la siguiente descripción de su funcionamiento:

- Dado que el ATmega2560 funciona a 16MHz, su ciclo de reloj tiene una duración de

$$T_{cpu} = \frac{1}{16 \times 10^6} = 62,5 \text{ ns.}$$

- Se emplea un *prescaler* de 256, para lo cual cada *falling edge* ocurre cada

$$T_{tick} = 256 T_{cpu} = 16 \mu s.$$

- Finalmente se emplea la cantidad de *falling edge* que existen en un segundo son

$$\text{ticks} = \frac{1}{16 \times 10^{-6}} = 62\,500$$

En tanto 62500 es el valor que se empleará para configurar la interrupción.

En cuanto a su codificación se presentan configuraciones relacionadas a los calculos hechas. Para visualizarlas refiérase a las siguientes líneas de código y sus comentarios.

```
1 void setup() {
2   // Todo apagado, modo normal registro A. Counter1 del ATMEGA2560
3   TCCR1A = 0b00000000;
4   // Todo apagado, modo normal registro B. Counter1 del ATMEGA2560
5   TCCR1B = 0b00000000;
6   // Configuración de preescaler a 256 (BIT CS12)
7   TCCR1B |= B00000100;
8   // Habilitar interrupción por comparación usando el registro TIMSK1 (modo CTC)
9   TIMSK1 |= B00000010;
10  // Establecer valor TOP para generar interrupción cada 1 segundo (valor a comparar calculado)
11  OCR1A = 62500/fs;
12  interrupts();
13 }
```

Una vez que se ha realizado la interrupción, el código de ejemplo se encarga del conteo de pulsos dentro de la misma. El conteo de pulsos se emplea para computar las revoluciones por minuto (rpm) del motor. Para llevar los pulsos a rpm se emplea la siguiente lógica:

- La cantidad de pulsos que conforman una vuelta son 36 (por los 36 hoyos del *encoder*); luego, cada segundo se cuenta la cantidad de pulsos que han sucedido en un segundo.
- Dada esa cantidad de pulsos, se genera una división para 36 a fin de hallar las vueltas que se han generado en un segundo.
- Finalmente, se multiplica las vueltas generadas en un segundo por sesenta: se hallan las revoluciones por minuto.

Tal lógica nos deja con la expresión

$$\text{RPM} = \frac{60 (\text{Pulsos contados})}{36}.$$

Para visualizar el conteo de los pulsos y conversión a RPM dentro del código de ejemplo refiérase a las siguientes líneas y a sus comentarios:

```
1 void setup() {
2   //Conteo de pulso cada vez que el pin digital detecta un hoyo del encoder
3   attachInterrupt(digitalPinToInterrupt(entrada), contarPulso, FALLING);
4 }
5
6 (...)
7
8 ISR(TIMER1_COMPA_vect) {
9   TCNT1=0; // Resetea el timer (Timer 1 a cero)
10  // Calcular RPM (pulsos por segundo / pulsos por revolución) * 60
11  rpm = (float(conteo_pulsos)*60)*fs / (PULSOS_POR_REV);
12  conteo_pulsos=0; // Resetea los pulsos
13 }
14
15 (...)
16
17 // Interrupción por Hardware para contar los pulsos del motor
18 void contarPulso(){
19   conteo_pulsos++; // Incrementar contador al detectar pulso
20 }
```

IV-F. Implementación del PID

La Figura 1 muestra el esquema del controlador a implementar en Labview. Este controlador PID será implementado en una de las interrupciones que ejecuta su interrupción cuando se haya cumplido el tiempo de muestreo escogido arbitrariamente cuando se definió la función de transferencia, en este caso 50ms.

Declaración de variables

```
1 float deltaU;           // Cambio en la señal de control
2 float u=0;              // Señal de control para PWM
3 volatile float e[3];    // Array para almacenar los valores de error del controlador PID
4 float Kp;               // Ganancia proporcional
5 float Ki;               // Ganancia integral
6 float Td;               // Tiempo derivativo
```

En esta sección se definen las variables fundamentales del controlador PID. El arreglo `e[3]` almacena los errores actuales y pasados. Las constantes `Kp`, `Ki` y `Td` representan las ganancias proporcional, integral y derivativa respectivamente, elementos esenciales del algoritmo PID.

Inicialización de parámetros PID en `setup()`

```
1 void setup() {
2   Kp = 0.01;
3   Ki = 0.015;
4   Td = 0;
5 }
```

Aquí se inicializan los parámetros del controlador. Se asignan valores pequeños a `Kp` y `Ki`, y se fija `Td` en cero, lo que indica que el componente derivativo no se está utilizando en esta configuración específica.

Control PID dentro de la rutina de interrupción ISR (`TIMER1_COMPA_vect`)

```
1 e[0]= slider_RPM - rpm;
2
3 deltaU += Kp * (e[0] - e[1]) + Ki * e[0] + Kp * (Td/T) * (e[0] - 2*e[1] + e[2]);
4
5 if (deltaU > 255)
6   deltaU = 255;
7 if (deltaU < 0)
8   deltaU = 0;
9
10 analogWrite(pin_motor, deltaU);
```

El controlador PID se ejecuta dentro de una interrupción de temporizador, lo cual garantiza una ejecución periódica, evitando así la latencia del sistema HMI.

Se calcula el error como la diferencia entre la referencia (`slider_RPM`) y la medida actual de velocidad (`rpm`). La ecuación PID discreta empleada corresponde a la forma incremental del controlador:

- $Kp * (e[0] - e[1])$: término proporcional incremental.
- $Ki * e[0]$: acumulación del error actual para el efecto integral.
- $Kp * (Td/T) * (e[0] - 2*e[1] + e[2])$: aproximación del término derivativo discreto.

El valor resultante `deltaU` es limitado entre 0 y 255, el rango permitido para la señal PWM. Finalmente, se aplica la salida al motor mediante `analogWrite`.

Esta estructura permite una regulación continua de la velocidad del motor, ajustando la señal de control en función de los errores presentes y pasados.

IV-G. Implementación con el HMI

Aquí tienes la explicación detallada sobre cómo se realiza la integración con el HMI (Human Machine Interface) usando la librería oficial de Stone y una librería personalizada, con los fragmentos de código destacados en la forma solicitada:

““latex

Inclusión de librerías y variables para temporización

```
1
2 #include "Stone_HMI_Define.h"
3 #include "Procesar_HMI.h"
4
5 unsigned long t_previo=0;
6 unsigned long t_previo1=0;
```

Se incluyen dos librerías esenciales: `Stone_HMI_Define.h`, que permite la comunicación directa con el HMI de Stone, y `Procesar_HMI.h`, una librería personalizada para procesar las respuestas del HMI de forma ordenada. También se definen dos variables para controlar la frecuencia de envío y recepción de datos con el HMI, lo cual es crucial para no saturar la comunicación serial.

Inicialización de comunicación y widgets del HMI

```
1
2 Serial.begin(115200);
3 Serial2.begin(115200);
4 STONE_push_series("line_series", "line_series1", 0);
5 STONE_push_series("line_series", "line_series2", 0);
6 STONE_push_series("line_series", "line_series3", 0);
7 Stone_HMI_Set_Value("slider", "slider1", NULL, 0);
8 HMI_init();
9 Stone_HMI_Set_Value("spin_box", "spin_box1", NULL, 1);
10 Stone_HMI_Set_Value("spin_box", "spin_box2", NULL, 15);
11 Stone_HMI_Set_Value("spin_box", "spin_box3", NULL, 0);
12 iniciarEstados();
```

En el `setup()`, se inicializa la comunicación serial con el PC y con el HMI mediante `Serial2`. Luego, se envían valores iniciales a tres gráficas tipo `line_series` en el HMI para establecer los ejes X. También se configuran widgets del HMI como un `slider` y tres `spin_box`, que se usan para ajustar los parámetros del controlador PID.

Lectura de parámetros del HMI en `loop()`

```
1
2 if (encendido_m == 1){
3 Kp = HMI_get_value("spin_box", "spin_box1") / 100;
4 Ki = HMI_get_value("spin_box", "spin_box2") / 1000;
5 Td = HMI_get_value("spin_box", "spin_box3");
6 }
7
8 slider_RPM= HMI_get_value("slider", "slider1");
```

Cada 50 ms, se leen los valores de los widgets del HMI: tres `spin_box` que ajustan los valores de K_p , K_i , y T_d , y un `slider` que define la referencia de velocidad del motor (`slider_RPM`). Esta dinámica permite que el usuario modifique en tiempo real el comportamiento del controlador PID desde el HMI.

Envío de datos hacia el HMI para visualización

```
1
2 dtostrf(slider_RPM, 7, 2, label2_text);
3 dtostrf(rpm, 7, 2, label4_text);
4 Stone_HMI_Set_Text("label", "label2", label2_text);
5 Stone_HMI_Set_Text("label", "label4", label4_text);
6
7
8 Cursor ->
9
10 STONE_push_series("line_series", "line_series1", slider_RPM);
11 STONE_push_series("line_series", "line_series2", rpm);
12 STONE_push_series("line_series", "line_series3", deltaU);
```


También cada 50 ms, el sistema convierte las variables `slider_RPM` y `rpm` a cadenas de texto y las muestra en etiquetas del HMI (`label2` y `label4`). Adicionalmente, se grafican en tiempo real la referencia de velocidad, la velocidad real y la señal de control mediante tres series de datos, lo cual permite al usuario observar de forma clara el desempeño del sistema de control.

Esta integración eficiente entre el microcontrolador y el HMI permite tanto la interacción como la visualización en tiempo real, ofreciendo una herramienta poderosa para la supervisión y ajuste de sistemas de control embebidos.

IV-H. Visualización de resultados

La Figura 7 muestra la salida del HMI tras ejecutar el programa y la interfaz en el HMI sobre el controllino. Como se puede observar, el valor de revoluciones por minuto (línea naranja) sigue correctamente a la referencia (línea azul). De igual manera, se grafica la señal de control en la gráfica rosa.

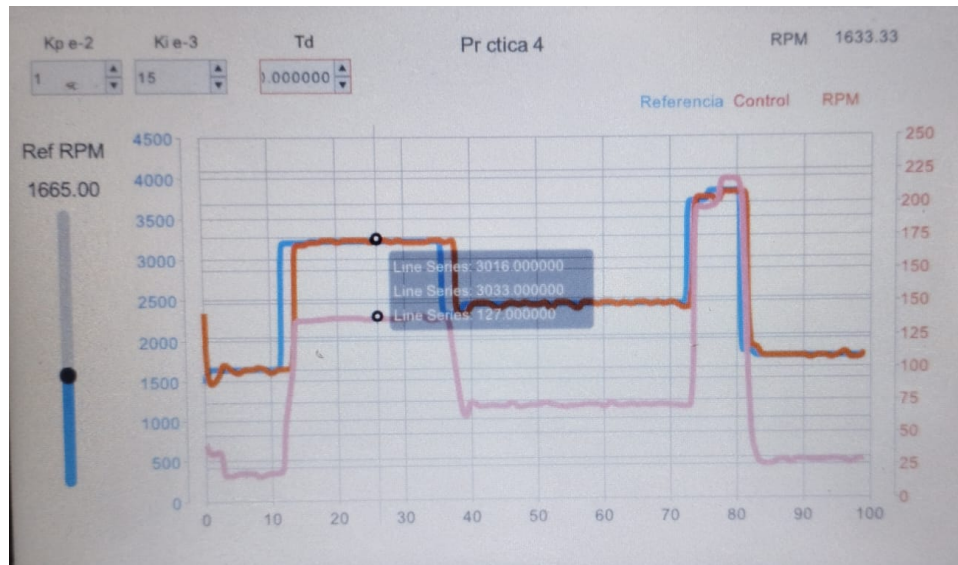


Figura 7. Interfaz de HMI en ejecución

V. CONCLUSIONES

1. Se demostró la correcta adaptación de las señales entre el EPC y el Controllino Mega. Se pudo adquirir correctamente los pulsos del encoder en la interrupción definida por hardware.
2. La configuración de dos interrupciones (externa para conteo de pulsos y por timer para cálculo de RPM) permitió medir con precisión la velocidad del motor y mantener una frecuencia de muestreo determinista.
3. Mediante la identificación en MATLAB se obtuvo la función de transferencia de primer orden, modelo que se ajusta a las características dinámicas del sistema.
4. La sintonización PID generó valores $K_p = 0$, $K_i = 0,02103$ y $K_d = 0$, logrando un control estable sin sobreimpulso significativo, sin embargo, estos valores tuvieron que ser ajustados manualmente debido a que el control no corregía correctamente el funcionamiento esperado del control.
5. La integración con la HMI Stone permitió la modificación en tiempo real de la referencia de velocidad y de las ganancias PID, así como la visualización inmediata de la señal de control y las RPM, validando la usabilidad y eficacia de la interfaz.
6. Los resultados experimentales confirmaron que el sistema alcanza la referencia de velocidad con un tiempo de asentamiento acorde al modelo identificado y mantiene un error estacionario inferior al 5 %.

REFERENCIAS

- [1] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [2] Departamento de Ingeniería Eléctrica, Electrónica y Telecomunicaciones, *Guía de uso de Tableros v2*, 1st ed., Ing. Henry Maldonado, Ed. Cuenca, Ecuador: Universidad de Cuenca, 2025.
- [3] H. Maldonado, *Puneros y retardos no bloqueantes*, presentación, Departamento de Ingeniería Eléctrica, Electrónica y Telecomunicaciones, Universidad de Cuenca, Cuenca, Ecuador, 2025.

VI. ANEXOS

```
1 //=====//
2 //      LIBRERAS      //
3 //=====//
4 #include <Controllino.h>
5 #include "Stone_HMI_Define.h" // Librería oficial de HMI Stone
6 #include "Procesar_HMI.h"     // Librería personalizada para procesar respuestas del HMI
7
8 //=====//
9 //      VARIABLES GLOBALES      //
10 //=====//
11
12 // PWM del motor
13 const int pin_motor = CONTROLLINO_D0; // Pin de salida PWM hacia el motor
14 int slider_RPM = 0; // Valor de referencia leido del slider del HMI
15 char label2_text[10]; // Texto que se mostrará en el label2 (duty cycle)
16
17 // Conteo de pulsos y cálculo de RPM
18 const int entrada = CONTROLLINO_IN1; // Pin de entrada de pulsos del sensor
19 volatile unsigned long conteo_pulsos = 0; // Contador de pulsos
20 char label4_text[10]; // Texto que se mostrará en el label4 (RPM)
21 float rpm = 0; // Valor actual de RPM
22 const uint16_t PULSOS_POR_REV = 36; // Pulsos por revolución del encoder
23 const float fs = 1 / 0.05; // Frecuencia de muestreo (Hz)
24
25 // PID Control
26 float deltaU = 0; // Cambio en la señal de control
27 float u = 0; // Señal de control aplicada al PWM
28 volatile float e[3]; // Array de errores: e[0] actual, e[1] anterior, e[2] dos
    atrs
29 float Kp; // Ganancia proporcional
30 float Ki; // Ganancia integral
31 float Td; // Tiempo derivativo
32 volatile float T = 0.05; // Periodo de muestreo (s)
33
34 // Control por botón físico
35 int estadoBoton_rojo = HIGH; // Estado actual del botón
36 int estadoAnterior_rojo = LOW; // Estado anterior del botón
37 int encendido_m; // Estado de encendido (0 = apagado, 1 = encendido)
38
39 // Máquina de estados para el motor
40 typedef enum {
41     APAGADO,
42     ENCENDIDO
43 } estados_t;
44 estados_t estado_actual;
45
46 void iniciarEstados() {
47     estado_actual = APAGADO;
48 }
49
50 // Temporización de actualización HMI
51 unsigned long t_previo = 0;
52 unsigned long t_previo1 = 0;
53
54 //=====//
55 //      DECLARACIÓN ISR      //
56 //=====//
57 void contarPulso(); // Prototipo de función de conteo de pulsos
58
59 //=====//
60 //      SETUP      //
61 //=====//
62 void setup() {
63     Serial.begin(115200); // Comunicación con PC
64     Serial2.begin(115200); // Comunicación con HMI
65
66     // Inicialización de gráficas en el HMI
67     STONE_push_series("line_series", "line_series1", 0);
68     STONE_push_series("line_series", "line_series2", 0);
69     STONE_push_series("line_series", "line_series3", 0);
70     Stone_HMI_Set_Value("slider", "slider1", NULL, 0);
71 }
```

```

72 // Parmetros PID iniciales
73 Kp = 0.01;
74 Ki = 0.015;
75 Td = 0;
76
77 // Configuracin de pines
78 pinMode(entrada, INPUT);
79 pinMode(pin_motor, OUTPUT);
80 attachInterrupt(digitalPinToInterrupt(entrada), contarPulso, FALLING);
81
82 // Configuracin de Timer1 para generar interrupcin peridica
83 noInterrupts();
84 TCCR1A = 0b00000000; // Modo normal
85 TCCR1B = 0b00000000; // Modo normal
86 TCCR1B |= B00000100; // Prescaler = 256
87 TIMSK1 |= B00000010; // Habilitar interrupcin por comparacin
88 OCR1A = 62500 / fs; // TOP para frecuencia deseada
89 interrupts();
90
91 // Inicializacin del procesamiento de HMI
92 HMI_init();
93 Stone_HMI_Set_Value("spin_box", "spin_box1", NULL, 1); // Kp
94 Stone_HMI_Set_Value("spin_box", "spin_box2", NULL, 15); // Ki
95 Stone_HMI_Set_Value("spin_box", "spin_box3", NULL, 0); // Td
96
97 iniciarEstados(); // Estado inicial
98 }
99
100 //=====//
101 // LOOP //
102 //=====//
103 void loop() {
104 // Leer parmetros cada 50 ms
105 if (millis() - t_previo1 >= 50) {
106     if (encendido_m == 0) {
107         deltaU = 0;
108         Kp = 0;
109         Ki = 0;
110         Td = 0;
111     }
112
113     if (encendido_m == 1) {
114         Kp = HMI_get_value("spin_box", "spin_box1") / 100.0;
115         Ki = HMI_get_value("spin_box", "spin_box2") / 1000.0;
116         Td = HMI_get_value("spin_box", "spin_box3");
117     }
118
119     slider_RPM = HMI_get_value("slider", "slider1");
120     t_previo1 = millis();
121 }
122
123 // Enviar valores al HMI cada 50 ms
124 if (millis() - t_previo >= 50) {
125     t_previo = millis();
126
127     dtostrf(slider_RPM, 7, 2, label2_text); // Conversin float a texto
128     dtostrf(rpm, 7, 2, label4_text);
129
130     Stone_HMI_Set_Text("label", "label2", label2_text);
131     Stone_HMI_Set_Text("label", "label4", label4_text);
132
133     STONE_push_series("line_series", "line_series1", slider_RPM);
134     STONE_push_series("line_series", "line_series2", rpm);
135     STONE_push_series("line_series", "line_series3", deltaU);
136 }
137 }
138
139 //=====//
140 // INTERRUPCIN POR TIMER1 //
141 //=====//
142 ISR(TIMER1_COMPA_vect) {
143     TCNT1 = 0; // Reiniciar contador
144
145     // Alternar estado de encendido con bot n f sico

```

```

146 estadoBoton_rojo = digitalRead(CONTROLLINO_I16);
147 if (estadoAnterior_rojo == HIGH && estadoBoton_rojo == LOW) {
148     encendido_m = (encendido_m == 0) ? 1 : 0;
149 }
150 estadoAnterior_rojo = estadoBoton_rojo;
151
152 // Calcular RPM
153 rpm = (float)(conteo_pulsos * 60.0 * fs) / PULSOS_POR_REV;
154 e[0] = slider_RPM - rpm;
155
156 // Ecuación PID en forma discreta (recurrencia)
157 deltaU += Kp * (e[0] - e[1]) + Ki * e[0] + Kp * (Td / T) * (e[0] - 2 * e[1] + e[2]);
158
159 // Saturación de la señal de control
160 if (deltaU > 255) deltaU = 255;
161 if (deltaU < 0) deltaU = 0;
162
163 analogWrite(pin_motor, deltaU); // Aplicar PWM
164
165 // Enviar datos por consola
166 Serial.print("\nRPM: ");
167 Serial.println(rpm);
168 Serial.print("Referencia: ");
169 Serial.println(slider_RPM);
170 Serial.print("Control: ");
171 Serial.println(deltaU);
172 Serial.print("Kp: ");
173 Serial.println(Kp);
174
175 // Actualizar historial de errores
176 e[2] = e[1];
177 e[1] = e[0];
178
179 conteo_pulsos = 0; // Reiniciar contador de pulsos
180 }
181
182 //=====//
183 //  INTERRUPTCIÓN POR PULSO //
184 //=====//
185 void contarPulso() {
186     conteo_pulsos++; // Incrementar conteo por cada pulso
187 }

```

Listing 1. Código correspondiente a la práctica 4

El siguiente [enlace](#) corresponde al repositorio que presenta código, archivos, documentación y evidencia del funcionamiento.



Figura 8. Repositorio de prácticas